King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 09

Heaps

Term 261

Binary Heaps

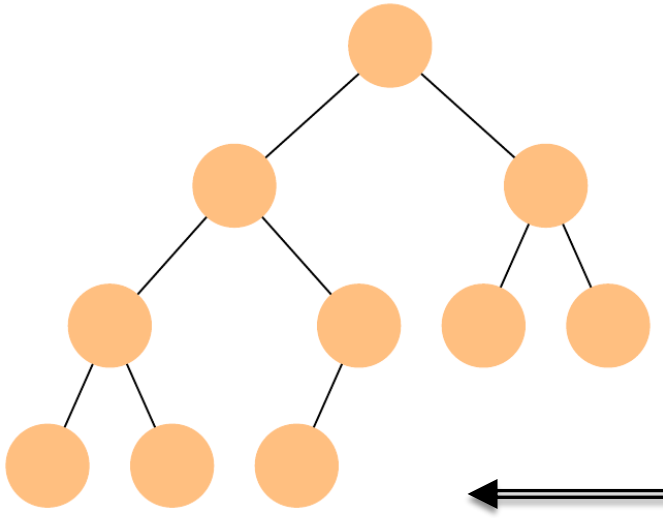
- What is a Binary Heap?
- Array representation of a Binary Heap
- MinHeap implementation
- Operations on Binary Heaps:
  - insert
  - deleting an arbitrary key
  - changing the priority of a key
  - deleteMin for minheap
  - deleteMax for maxHeap
- Building a binary heap (Heapify)
  - Top-down approach
  - Bottom-up approach
- Heap sort
- Heap as a priority queue

What is a Binary Heap?

Max Heap: The value of each node is greater than its children.

Min Heap: The value of each node is Smaller than its children.

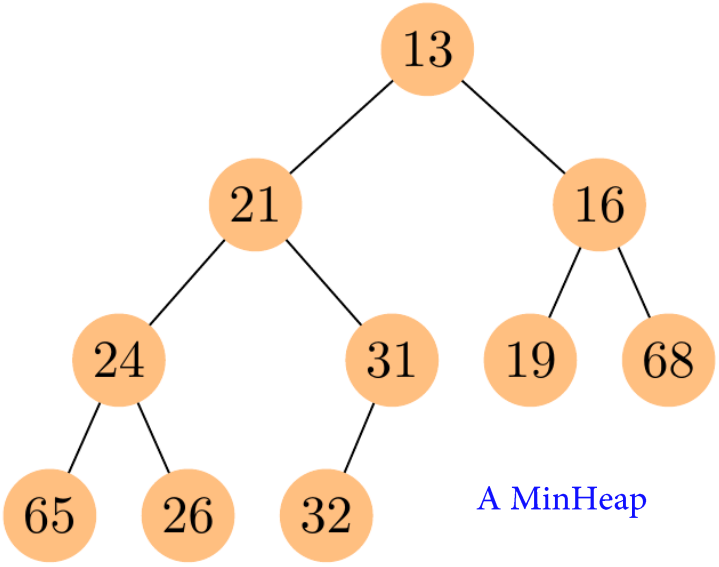
- A binary heap is a complete binary tree with one of the following heap order properties:
  1. MinHeap property: Each node must have a key that is $\leq$ the key of its children.
  2. MaxHeap property: Each node must have a key that is $\geq$ the key of its children.
  - A binary heap with all keys equal is both a MinHeap and a MaxHeap.



All levels except the bottom one must be fully populated with nodes

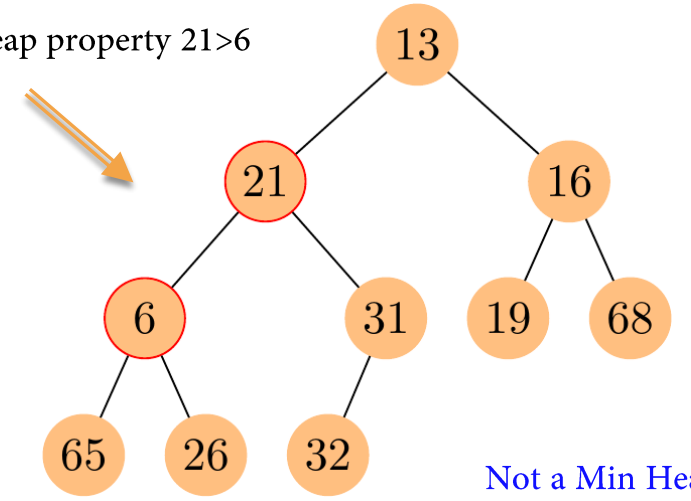
All missing nodes, if any, must be on the right side of the lowest level

MinHeap and non-MinHeap examples

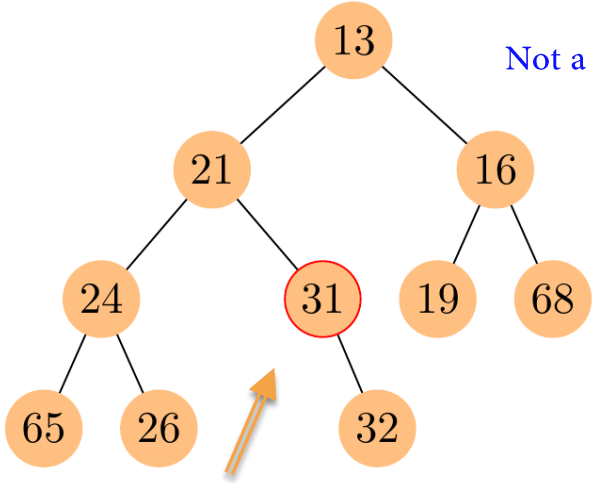


A MinHeap

Violates MinHeap property 21>6

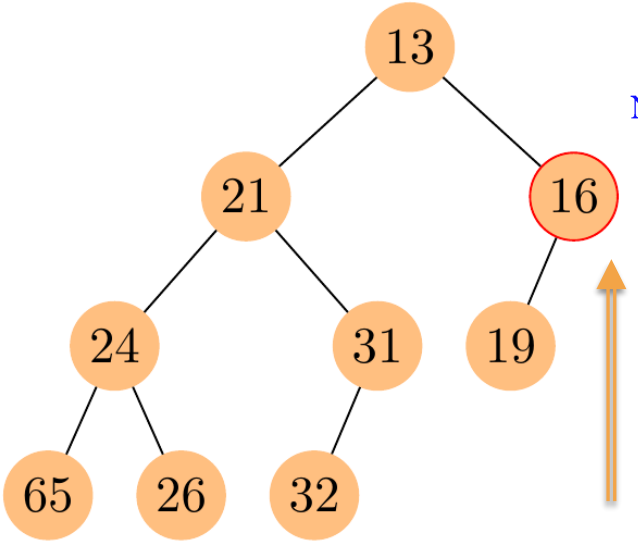


Not a Min Heap



Not a Heap

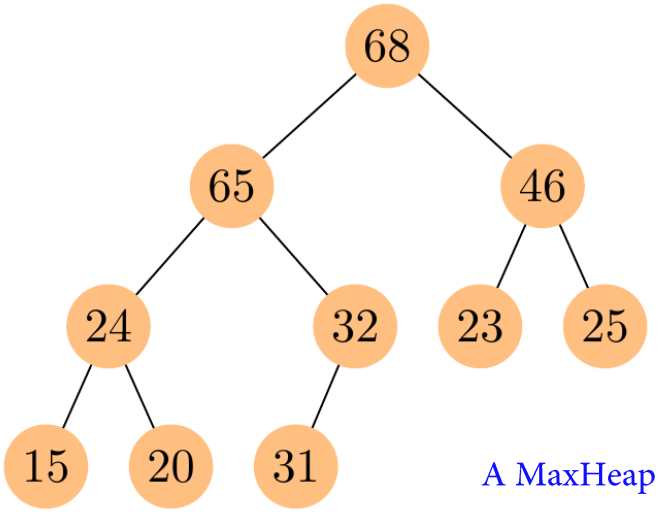
Violates heap structural property



Not a Heap

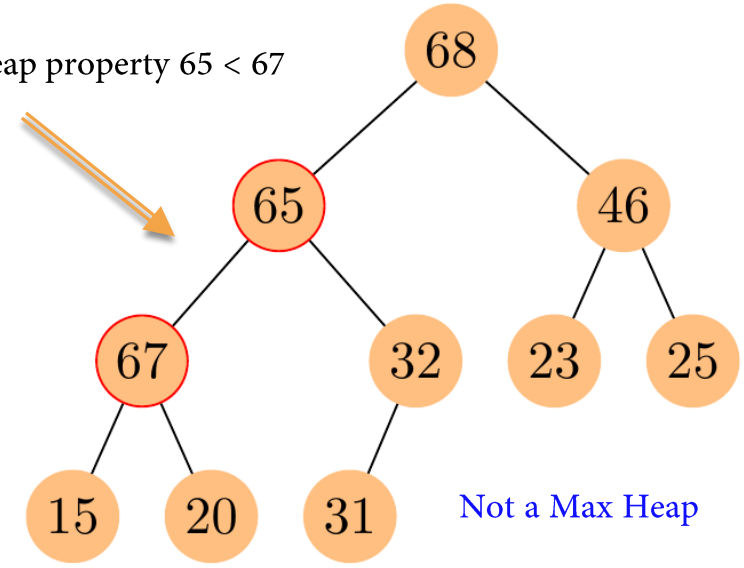
Violates heap structural property

MaxHeap and non-MaxHeap examples

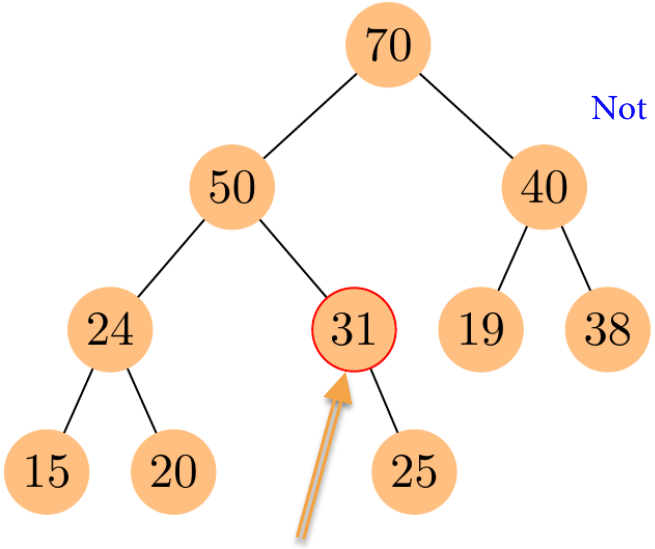


A MaxHeap

Violates MaxHeap property 65 < 67



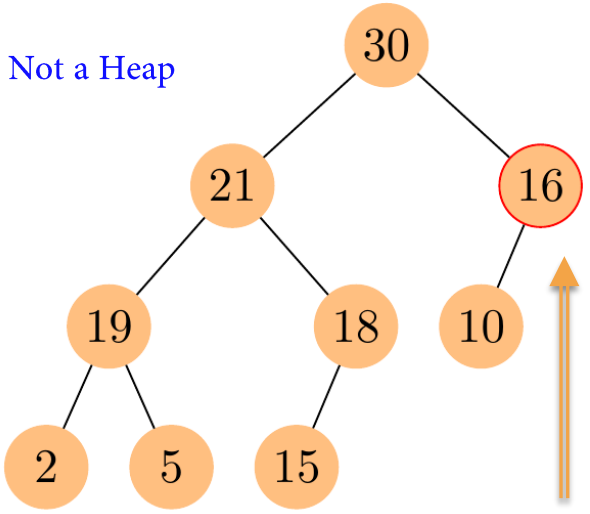
Not a Max Heap



Not a Heap

Violates heap structural property

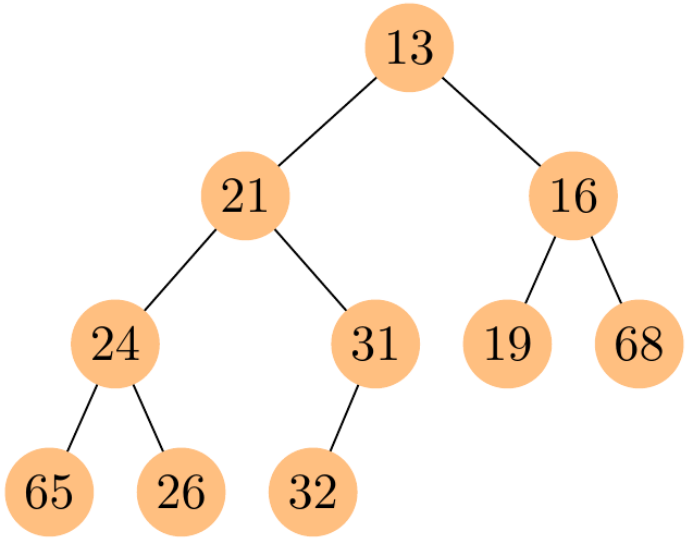
Not a Heap



Violates heap structural property

Array Representation of a Binary Heap

- A heap is a binary tree data structure that is represented and manipulated more efficiently using an array (1-indexed)
  - Since a heap is a complete binary tree, its node values can be stored in an array, without any gaps, in a breadth-first order, where: Value(node i) IS STORED AT array[i], for $i \geq 1$



	13	21	16	24	31	19	68	65	26	32
0	1	2	3	4	5	6	7	8	9	10

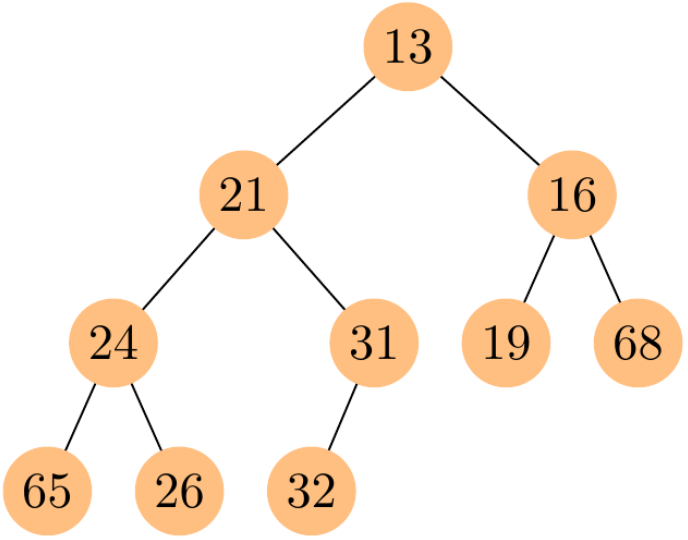
```
public class BinaryHeap<T extends Comparable<T>>{
   // private int capacity = 10;

   private int heapSize;   // Number of elements in heap
   private T[] heap;       // The heap array

   public BinaryHeap(int capacity)   {
      heapSize = 0;
      heap = (T[]) new Comparable[capacity + 1];
      this.capacity = capacity ;
   }
```

Array Representation of a Binary Heap (contd.)

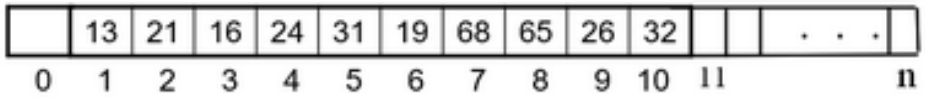
- The representation of a binary heap in an array makes several operations very fast:
  - add a new node at the end: O(1)
  - from a node, find its parent: O(1)
  - swap parent and child: O(1)
  - a lot of dynamic memory allocation of tree nodes is avoided

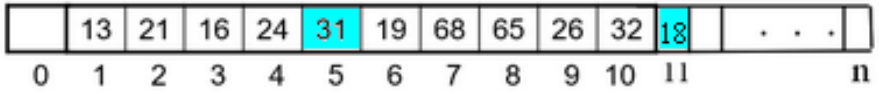


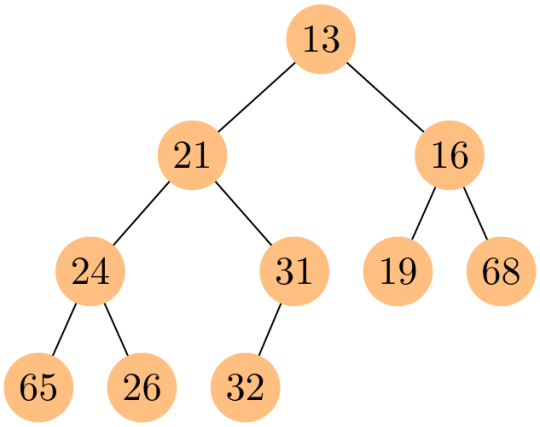
	13	21	16	24	31	19	68	65	26	32
0	1	2	3	4	5	6	7	8	9	10

- The root is array[1].
- If a node at position i,
  - The left child, if any, is at position 2i.
  - The right child, if any, is at position 2i+1.
  - The parent at position $\left\lfloor \frac{i}{2} \right\rfloor$, where i > 1

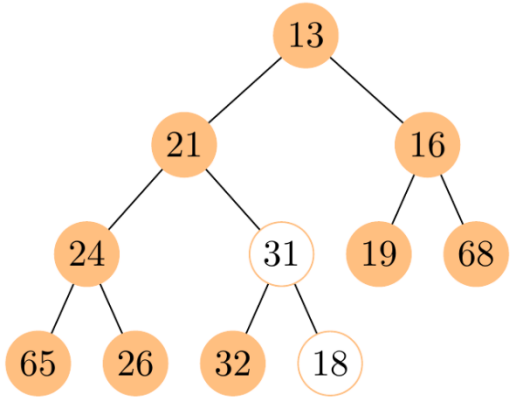
MinHeap Insertion Example



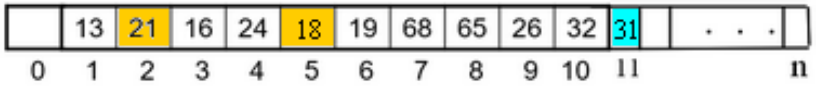


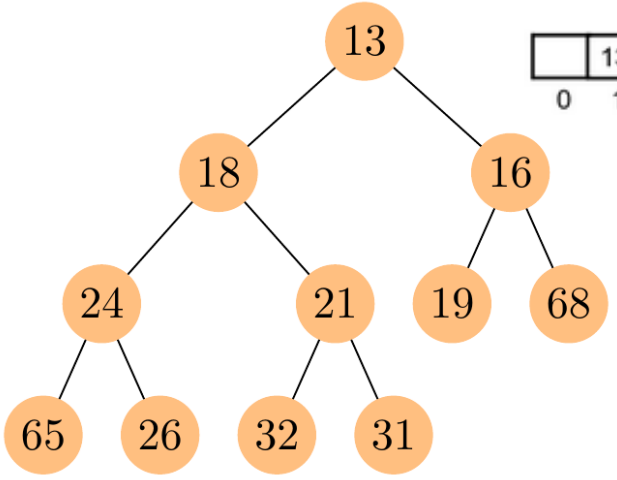


Insert 18

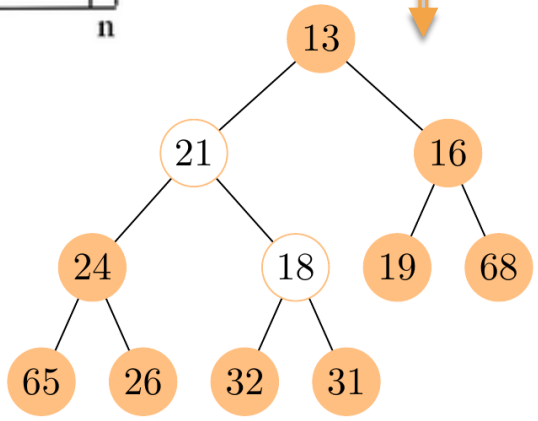


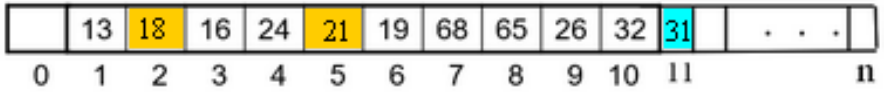
Percolate up

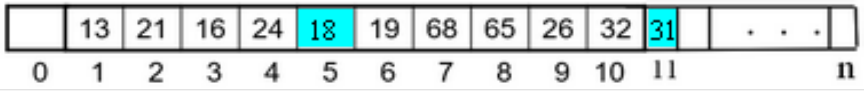




Percolate up







MinHeap Insertion

- The pseudo code algorithm for inserting a key in a MinHeap is:

```
insert(key){
   if(the heap is not full)
     insert key at the end of the heap ;
     percoluateUP the key
}
```

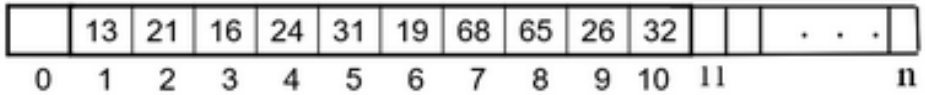
```
insert( key){
    heapSize =  heapSize + 1;
    A[heapSize] = key;
    percolateUp(heapSize)
 }
```

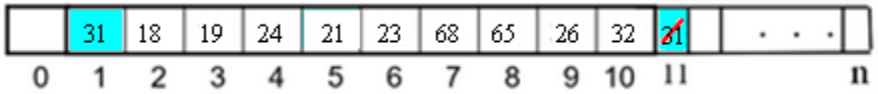
- The process of swapping an element with its parent, in order to restore the heap order property is called percolate up, sift up, or reheapification upward.

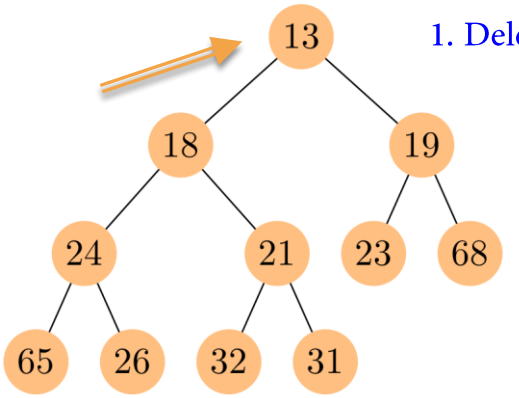
//Complexity: O(log n)

```
percolateUp(int i)
    while( i > 1 &&  heap[i] < heap[i/2] ){
         temp = heap[i/2];
         heap[i/2] = heap[i];
         heap[i] = temp;
         i = i/2;
     }
 }
```

MinHeap delete Example



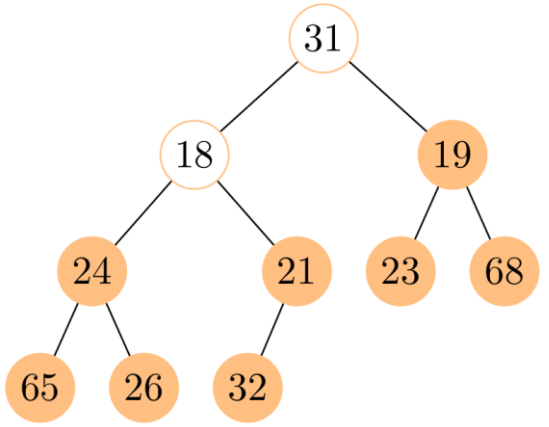




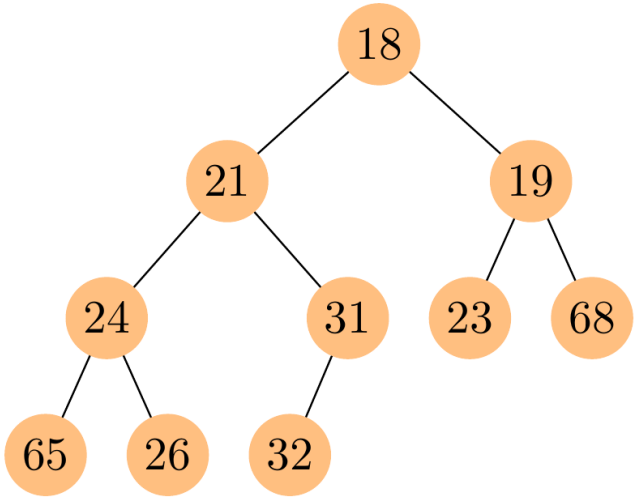
1. Delete 13, min element,

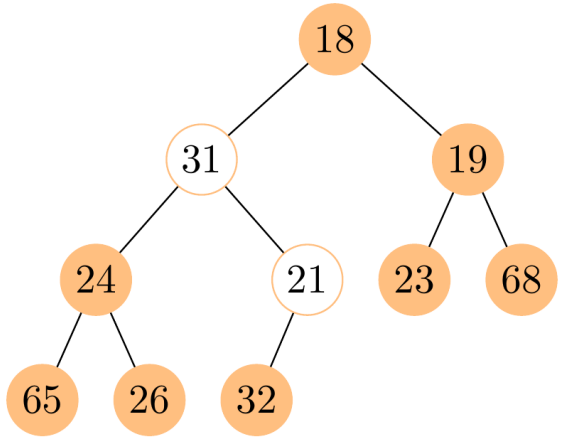
2. Replace by value of last node

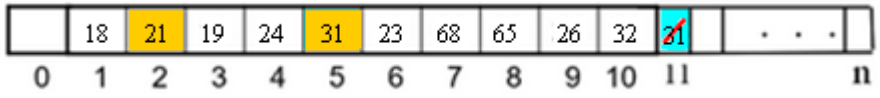
3.delete last node

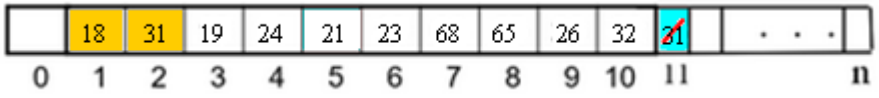


Percolate down









Heap delete

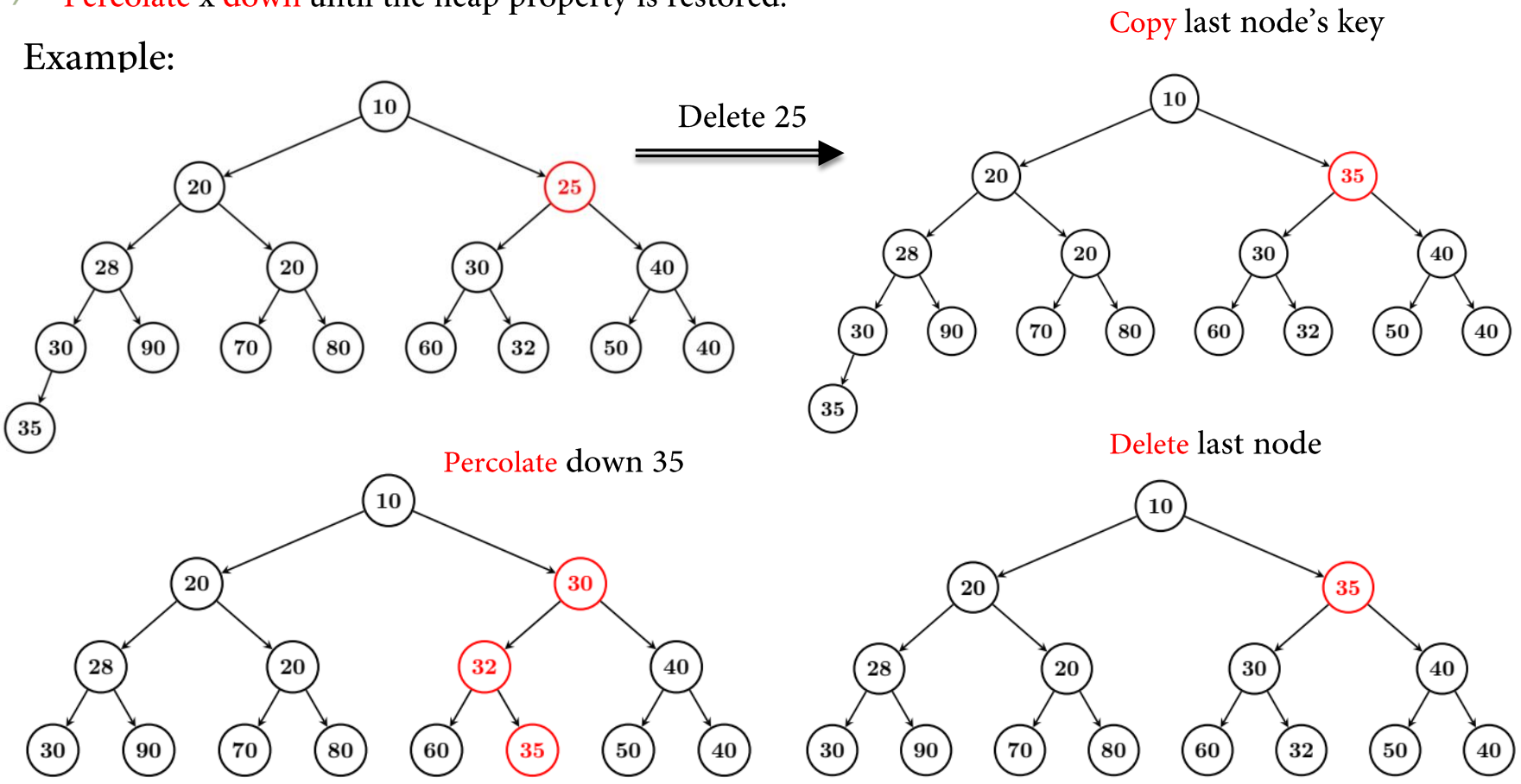
- The process of swapping an element with its child, in order to restore the heap order property is called percolate down, sift down, or reheapification downward.
- Thus, the steps for deletion are:
  - Replace the key at the root by the key of the last leaf node.
  - Delete the last leaf node.
  - As long as the heap order property is violated, percolate down.

```
deleteMin(){// DeleteMin for minheap or DeleteMax for max-heap
        if (heapSize == 0) return;
        key = array[1];
        array[1] = array[heapSize];
        heapSize =  heapSize - 1;
       percolatedown(1);
}
```

```
percolateDown(i) {
    temp = heap[i]; // Store current value
    done = false;
     while (!done && 2 * i <= heapSize) { // While node has at least one child
           child = 2 * i; // Left child
           // If right child exists and is smaller than left child
            if (child + 1 <= heapSize && heap[child + 1] < heap[child]) {
                  child++;  // 2i +1
            }
            // If smallest child is smaller than current value
             if (heap[child] < temp) {
                   heap[i] = heap[child]; // Move child up
                   i = child; // Move down
             } else { done = true; }
      }
   heap[i] = temp; // Place original value in correct position
}
```

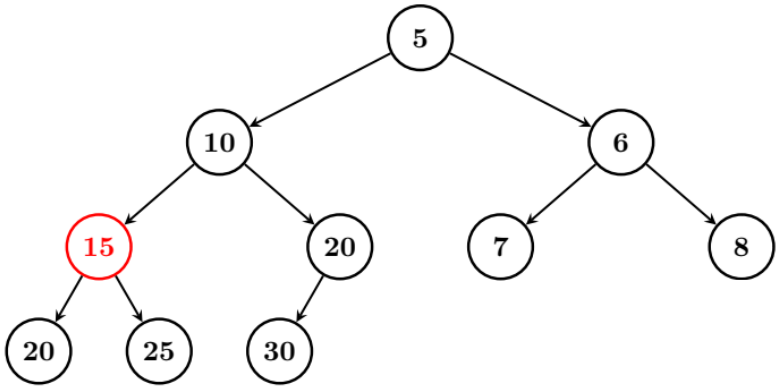
Deleting an arbitrary key

- The algorithm of deleting an arbitrary key from a heap is:
  - Copy the key x of the last node to the node containing the deleted key.
  - Delete the last node.
  - Percolate x down until the heap property is restored.
- Example:



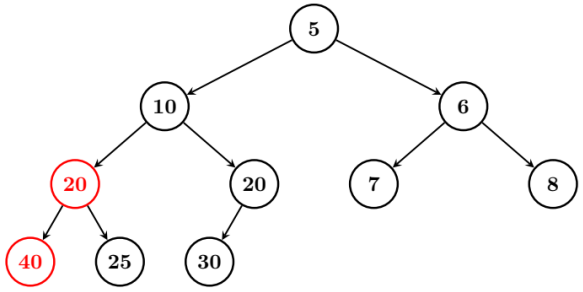
Changing the priority of a key

- There are three possibilities when the priority of a key x is changed:
  - The heap property is not violated. No percolation is needed.
  - The heap property is violated, and x has to be percolated up to restore the heap property.
  - The heap property is violated, and x has to be percolated down to restore the heap property.
- Example:



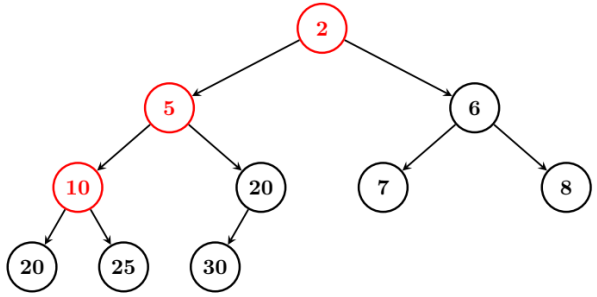
Change 15 to 40

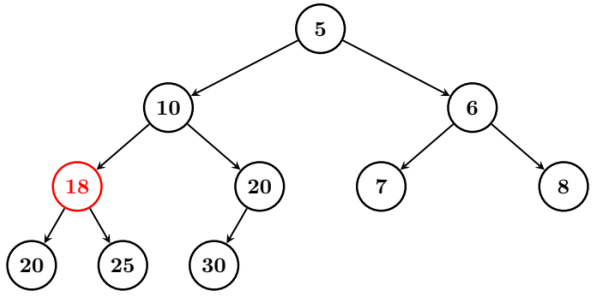
Percolate Down



Change 15 to 2

Percolate Up





Change 15 to 18

No Percolation

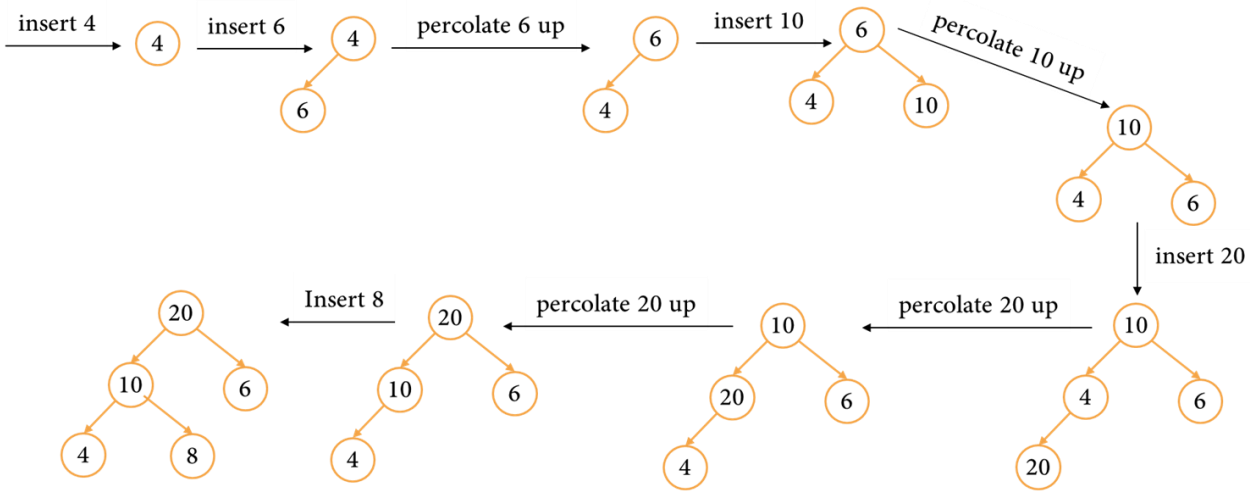
Building a heap

- A heap is built by inserting one key at a time into an initially empty heap.
- After each key insertion, if the heap property is violated, it is restored by percolating the inserted key upward.

The algorithm is:

```
for(int i=1; i <= heapSize; i++){
      read  key;
      binaryHeap.insert(key);
}
```

Example: Insert the keys 4, 6, 10, 20, and 8 in this order in an originally empty max-heap



Heap Construction

- The process of converting an arbitrary array (a complete binary tree) into a binary heap
- Two main approaches:
  - Top-down : Start from the second node and percolate up each node to fix the heap.
  - Bottom-up : Start from the last non-leaf node and percolate down each node to fix the heap.

- Top-down
  - The procedure in slide 14 can be used to convert an array into a Binary-heap;
  - We consider the array elements as if they are inserted one at a time, starting from the second element to the last.
- If the array indexes start at 1, the algorithm is:

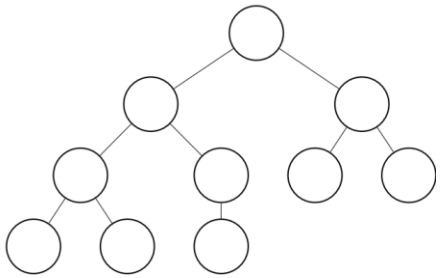
```
for(int i = 2; i <= n ;i++){
     percolateUP(i)
}
```

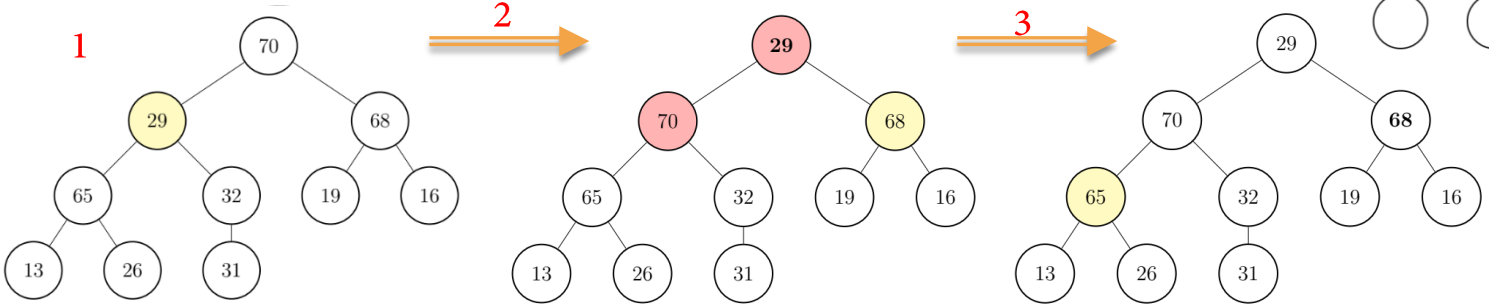
Complexity: O(n log n)

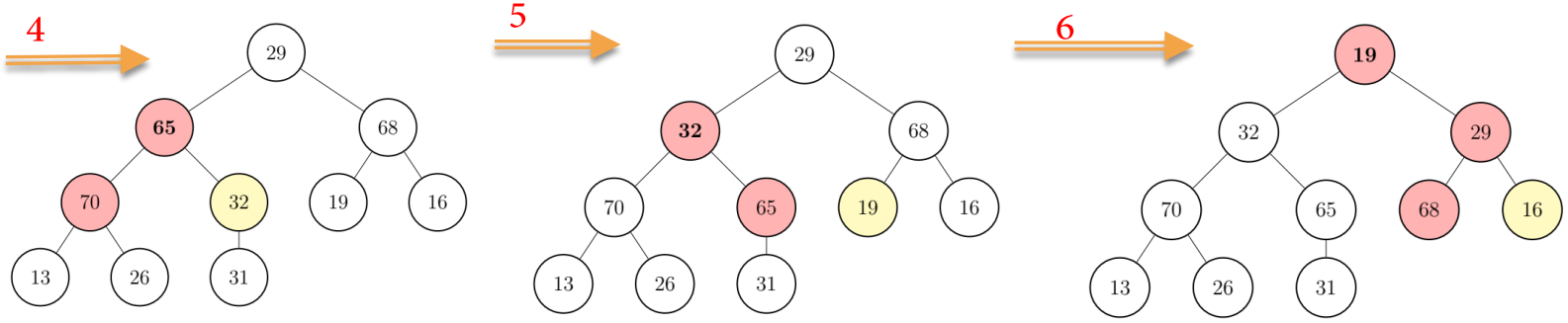
Complexity: O(n log n)

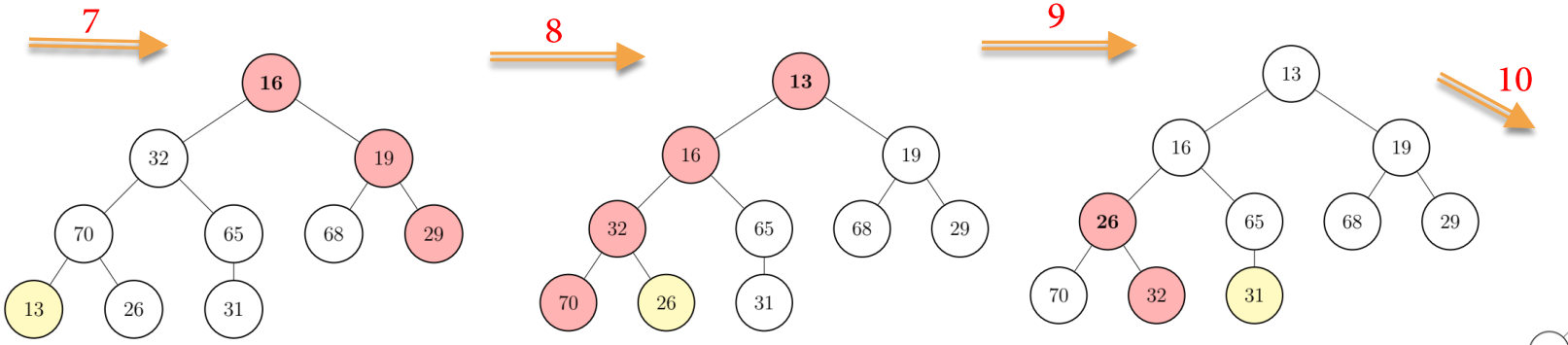
Top down : Converting an array into a MinHeap

70	29	68	65	32	19	16	13	26	31



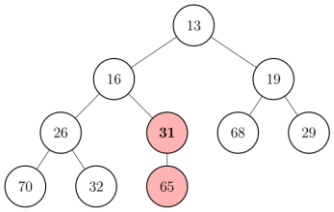






13	16	19	26	31	68	29	70	32	65

11



Bottom-up

- A bottom-up algorithm to convert an array into a binary heap:
  1. Start from the **last non-leaf node** in the array.
  2. For each node from $\lfloor n/2 \rfloor$ down to 1, perform **percolateDown**.
     1. Make the subtree rooted at the last non-leaf node a heap by invoking percolateDown.

```
public BinaryHeap(T[] array) {

   heapSize = array.length;
   this.capacity = heapSize;
   heap = (T[]) new Comparable[capacity + 1];

   for(int i = 0; i < array.length; i++)
        heap[i + 1] = array[i];

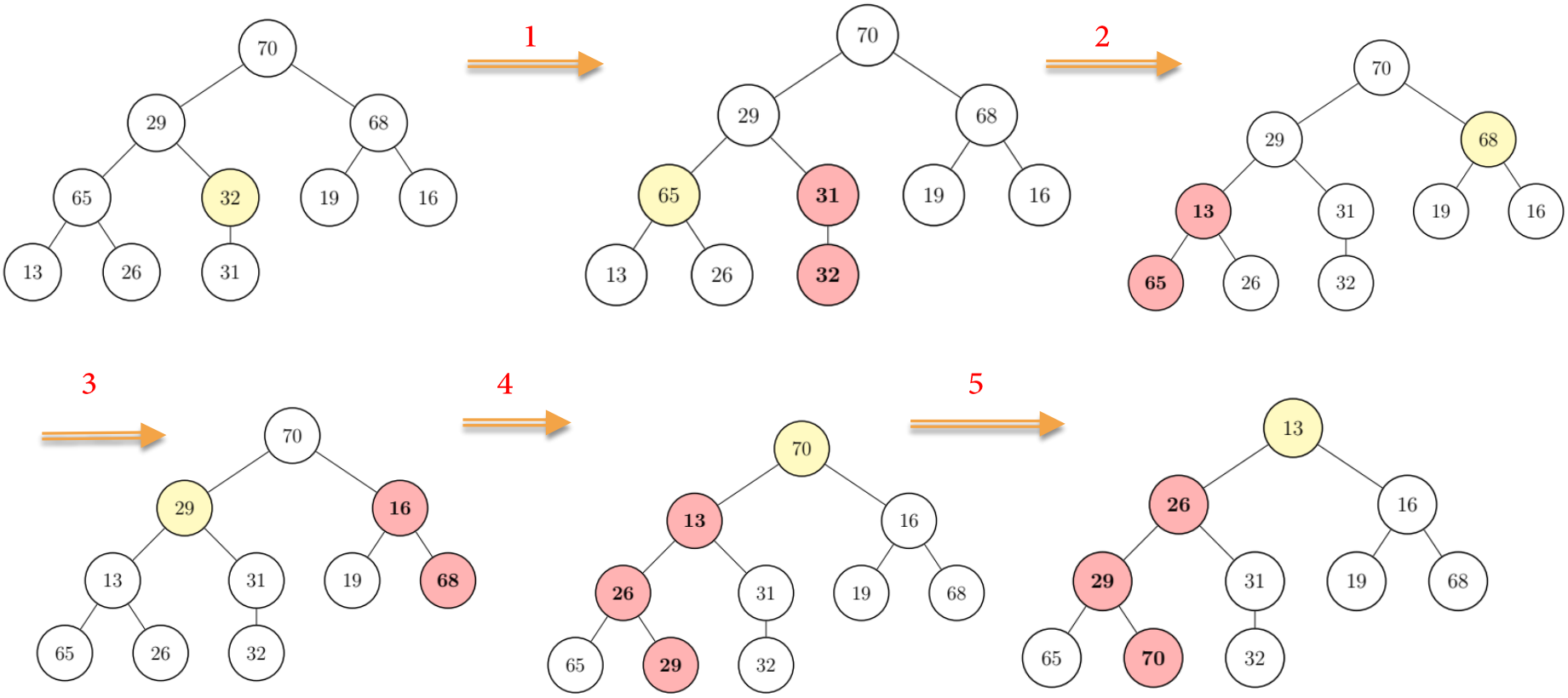
   buildHeapBottomUp();

}

private void buildHeapBottomUp(){
    for(int i = heapSize / 2; i >= 1; i--)
          percolateDown(i);
}
```

Complexity: O(n)

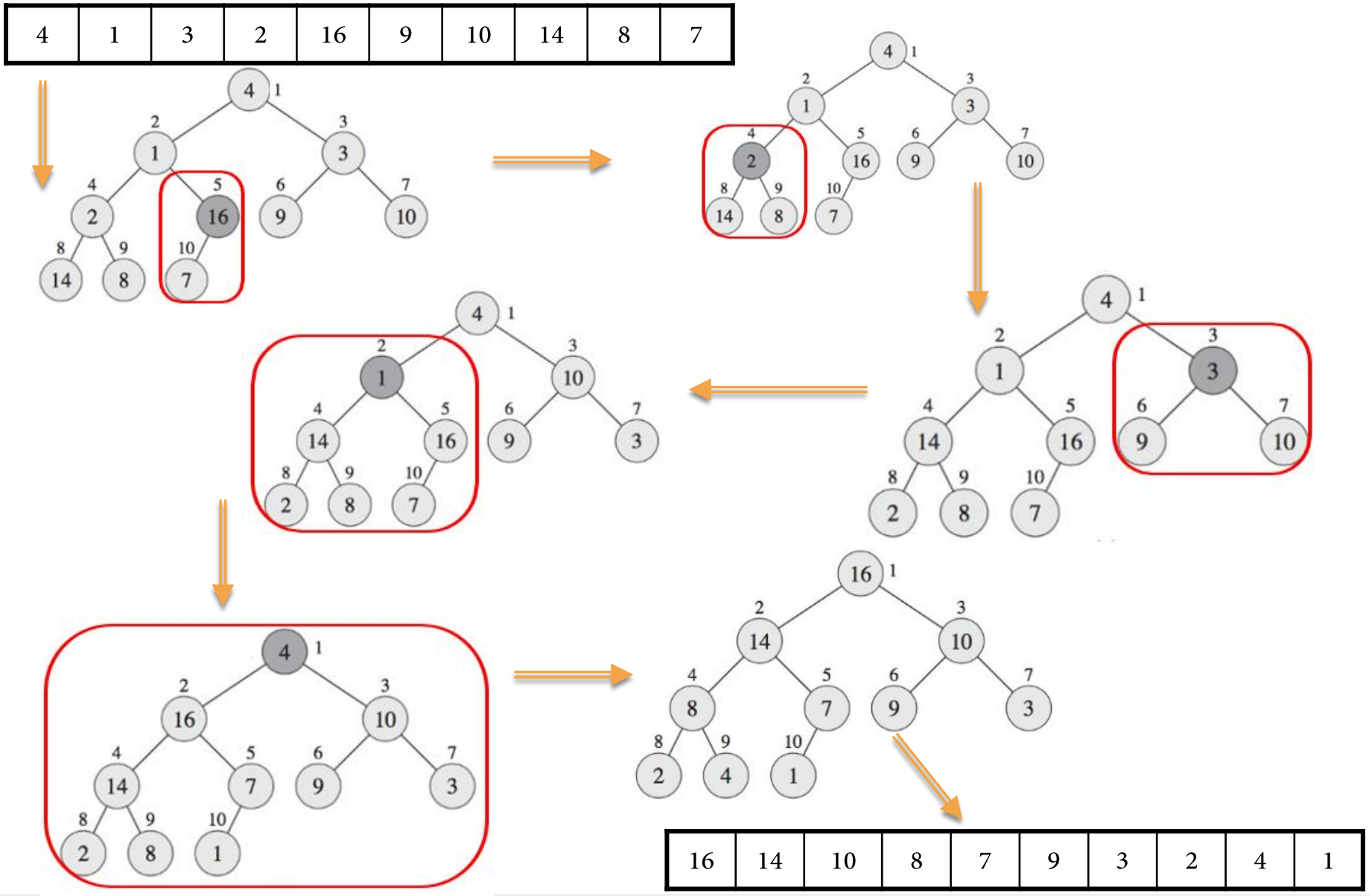
Bottom UP : Converting an array into a MinHeap



At each stage convert the highlighted tree into a minHeap by percolating down starting at the root of the highlighted tree (yellow node).

Bottom UP : Converting an array into a MaxHeap

At each stage convert the highlighted tree into a MaxHeap by percolating down starting at the root of the highlighted tree.



Heap Application: Heap Sort

- A MinHeap or a MaxHeap can be used to implement an efficient sorting algorithm called Heap Sort.
- Heap sort steps:
  - Build a min [or max] heap from an unsorted array.
  - Remove the current minimum [or maximum] from the heap n times and store in an array.
- The following algorithm uses a MinHeap:

```
public  static void  heapSort(Comparable[] array){
   BinaryHeap  heap  =  new BinaryHeap(array, array.length) ;
   for(int i = 0; i < array.length; i++)
      array[i] = heap.deleteMin() ;
}
```

- Because the deleteMin algorithm is O(log n), heapSort is an O(n log n) algorithm.
- Apart from needing the extra storage for the heap, heapSort is among efficient sorting algorithms.

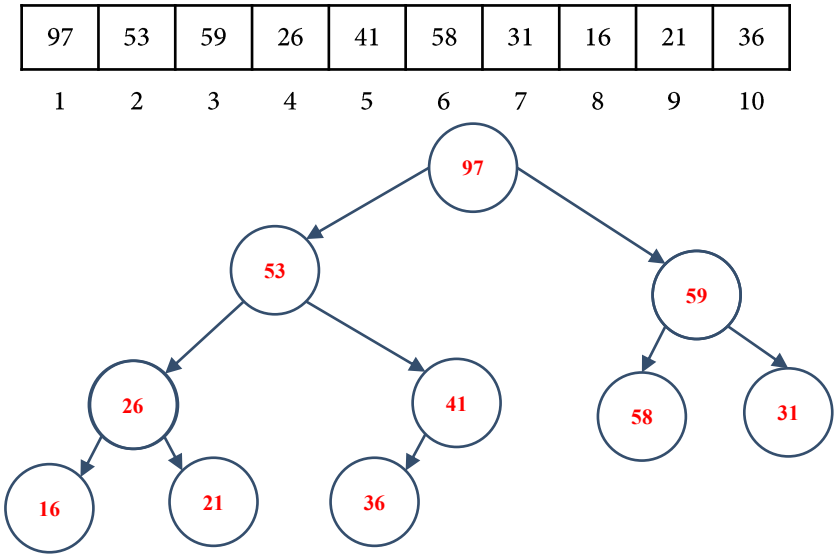
Heap Sort with no extra storage

- Observation: after each deleteMin, the size of heap shrinks by 1
- We can use the last cell just freed up to store the element that was just deleted
- After the last deleteMin, the array will contain the elements in decreasing order
- To sort the elements in the decreasing order, use a min heap
- To sort the elements in the increasing order, use a max heap

Heap Sort Algorithm

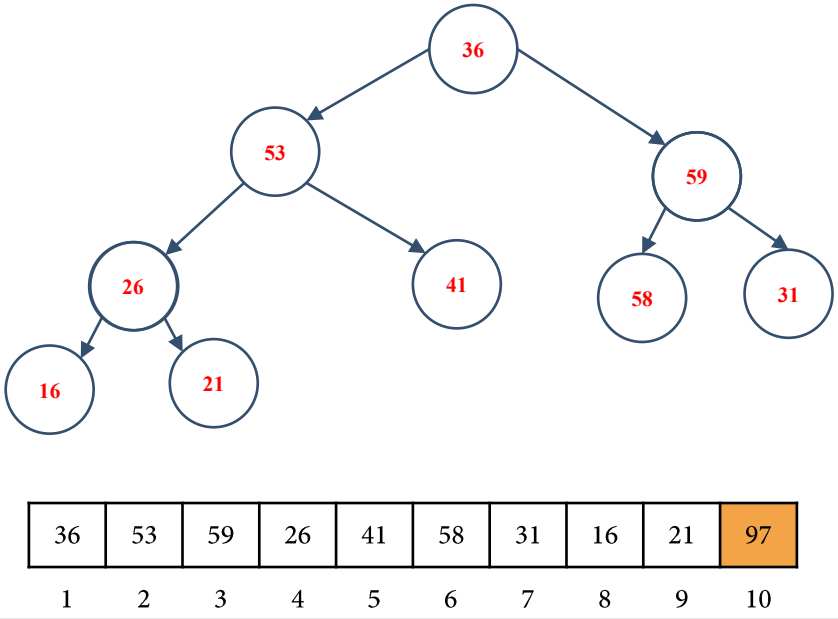
- First, convert the array into a heap using the **`percolateDown`** algorithm.
- Swap the root element with the element at the last position of the heap.
- Reduce the heap size by 1
- **`percolateDown`** the new root element again and repeat the process till the size of the heap is greater than 1.

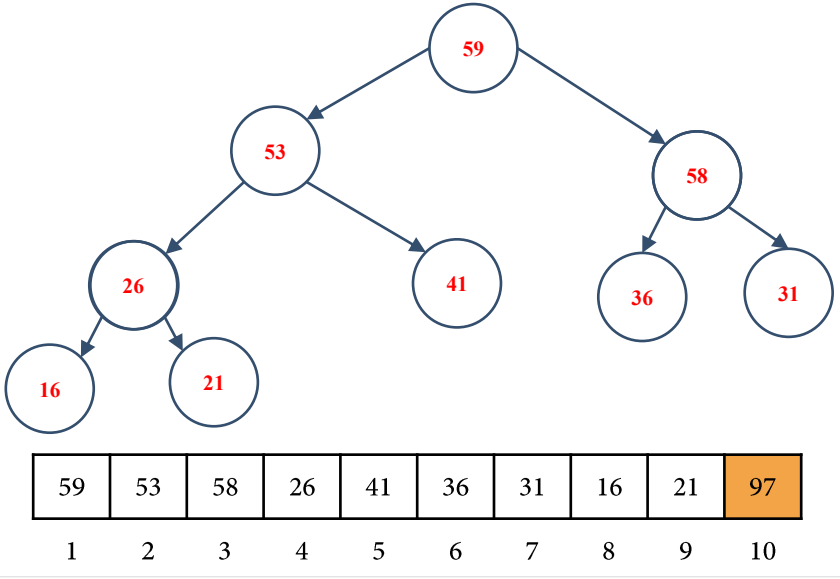
Heap sort



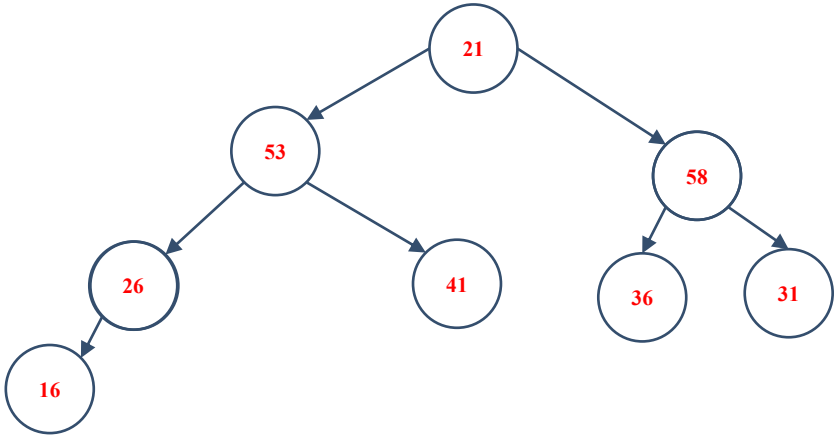
Example: Max heap after the buildHeap phase for the input sequence:

59, 36, 58, 21, 41, 97, 31, 16, 26, 53

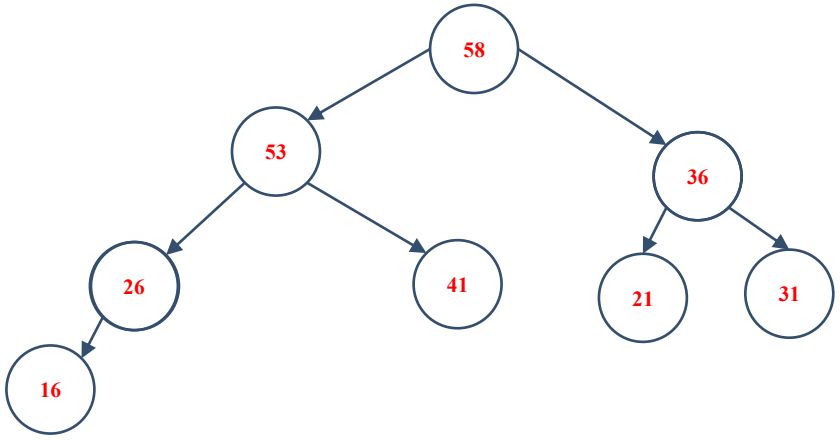




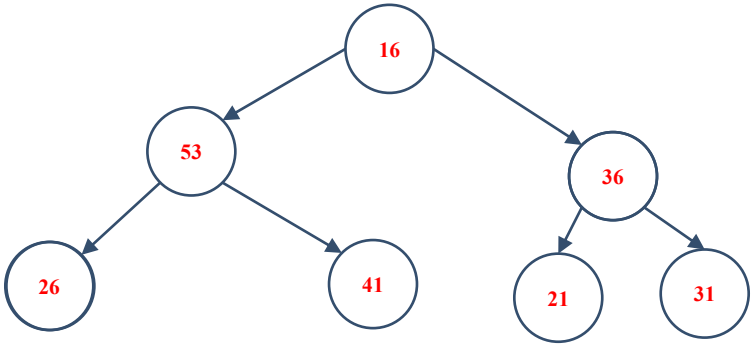
Heap sort…



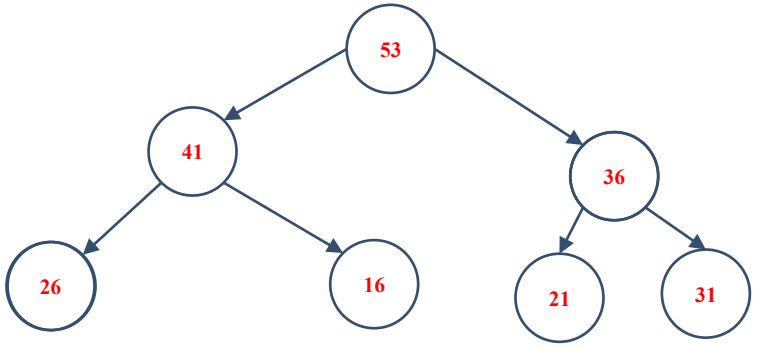
21	53	58	26	41	36	31	16	59	97
1	2	3	4	5	6	7	8	9	10



58	53	36	26	41	21	31	16	59	97
1	2	3	4	5	6	7	8	9	10

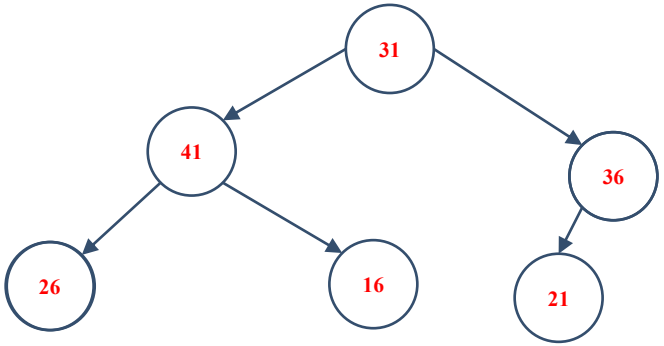


16	53	36	26	41	21	31	58	59	97
1	2	3	4	5	6	7	8	9	10

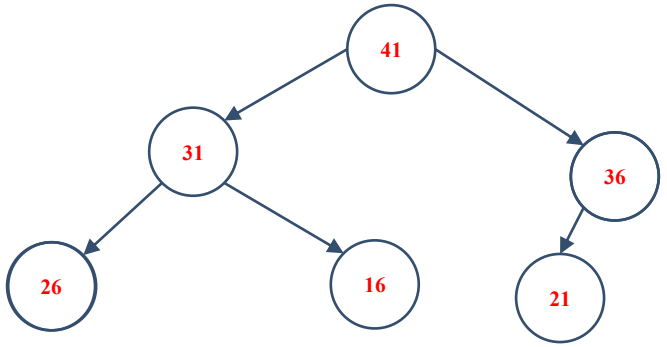


53	41	36	26	16	21	31	58	59	97
1	2	3	4	5	6	7	8	9	10

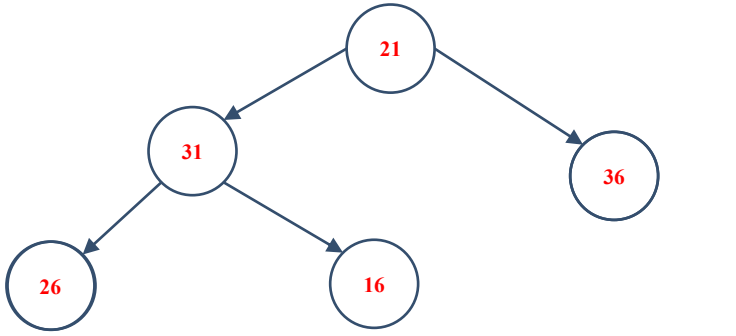
Heap sort…



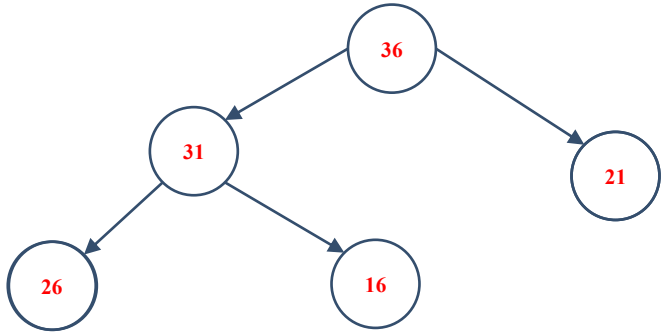
31	41	36	26	16	21	53	58	59	97
1	2	3	4	5	6	7	8	9	10



41	31	36	26	16	21	53	58	59	97
1	2	3	4	5	6	7	8	9	10

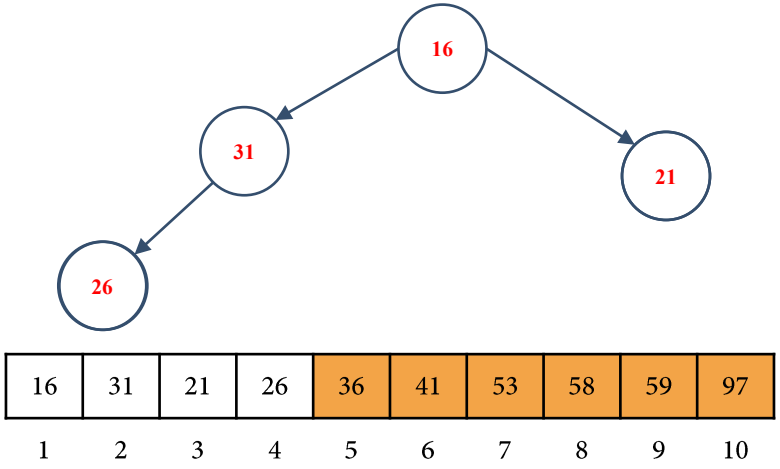


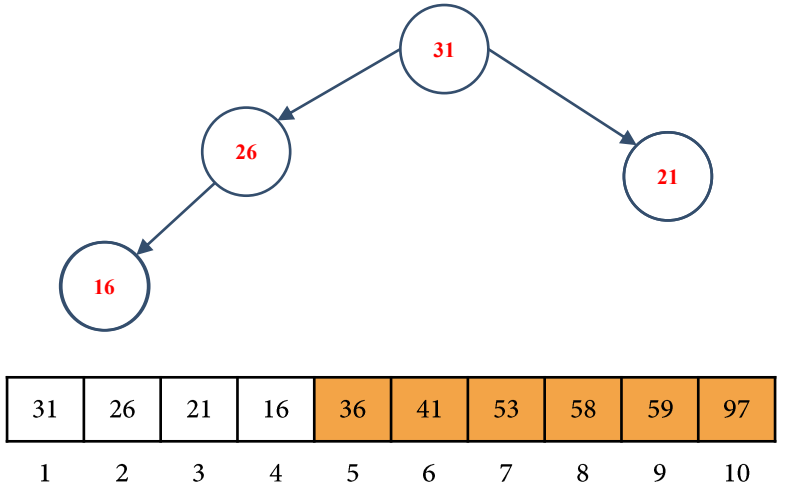
21	31	36	26	16	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10

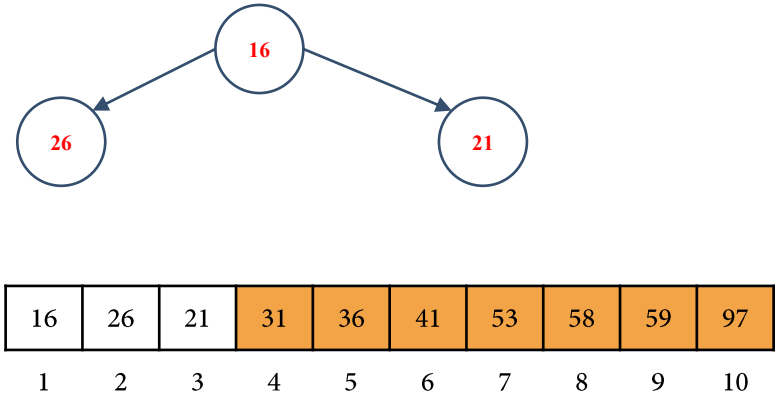


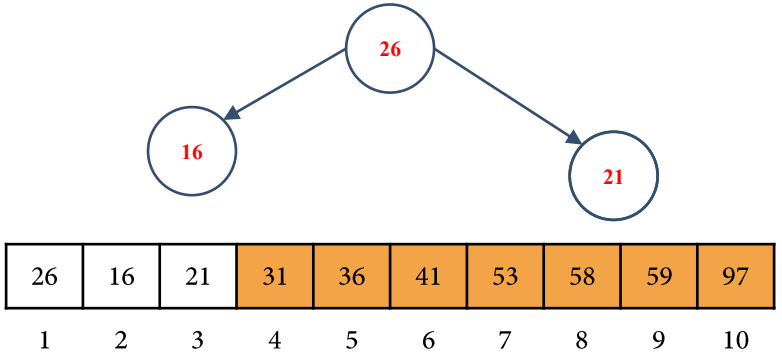
36	31	21	26	16	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10

Heap sort…

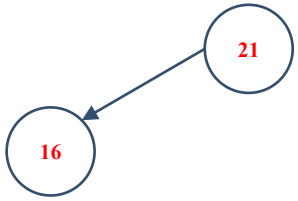

16
31
21
26
16 31 21 26 36 41 53 58 59 97
1 2 3 4 5 6 7 8 9 10


31
26
21
16
31 26 21 16 36 41 53 58 59 97
1 2 3 4 5 6 7 8 9 10

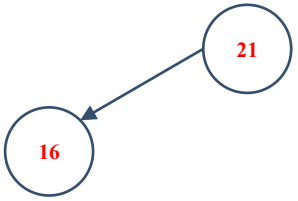

16
26
21
16 26 21 31 36 41 53 58 59 97
1 2 3 4 5 6 7 8 9 10


26
16
21
26 16 21 31 36 41 53 58 59 97
1 2 3 4 5 6 7 8 9 10

Heap sort…



21	16	26	31	36	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10



21	16	26	31	36	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10



16	21	26	31	36	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10

16	21	26	31	36	41	53	58	59	97
1	2	3	4	5	6	7	8	9	10

Heap Applications: Priority Queue

- A heap can be used as the underlying implementation of a priority queue.
  - A priority queue is a data structure in which the items to be inserted have associated priorities.
  - Items are removed from a priority queue in order of their priorities, starting with the highest priority item first.
  - Minimum priority queues treat low-valued keys as high-priority; maximum priority queues prioritizes high-valued keys
  - Priority queues are often used in resource management, simulations, and in the implementation of some algorithms (e.g., some graph algorithms, some backtracking algorithms).
- Several data structures can be used to implement priority queues. Below is a comparison of some:

Data structure	Enqueue	Find Min	Dequeue Min
Unsorted List	O(1)	O(n)	O(n)
Sorted List	O(n)	O(1)	O(1)
AVL Tree	O(log n)	O(log n)	O(log n)
MinHeap	O(log n)	O(1)	O(log n)